Кременчуцький національний університет імені Михайла Остроградського
(повне найменування вищого навчального закладу)

Кафедра комп'ютерної інженерії та електроніки
(повна назва кафедри)

ЛАБОРАТОРНА РОБОТА № 7

Тема: Проєктування функцій, робота з просторами імен та рекурсивні алгоритми

Мета роботи: Опанувати практичні навички процедурної та функціональної декомпозиції програмного коду мовою Python. Навчитися проектувати функції з адаптивними сигнатурами, використовуючи механізми пакування та розпакування позиційних (*args) та іменованих (**kwargs) аргументів, вивчити правила ізоляції та дозволу областей видимості змінних за моделлю LEGB. Опанувати синтаксис анонімних lambda-функцій для кастомного впорядкування складних інженерних колекцій у функціях вищого порядку (sorted, map, filter), а також розробити та дослідити рекурсивні алгоритми типу «розділяй і володарюй» із програмним контролем глибини системного стека викликів.

Студента III курсу КІ-24-1 групи
Ступінь вищої освіти бакалавр
(бакалавр, магістр)
Спеціальність 123 – «Комп'ютерна інженерія»
Освітня програма Комп'ютерна інженерія
Яцентюка Євгенія Олександровича

м. Кременчук 2026 рік

1. Мета роботи та математична постановка задачі

Мета роботи: опанувати процедурну та функціональну декомпозицію коду мовою Python; проєктувати функції з варіативними сигнатурами (*args, **kwargs), значеннями за замовчуванням та валідацією; розуміти правило видимості LEGB і замикання; застосовувати lambda-вирази як ключі багатокритеріального сортування та функції вищого порядку; розробити рекурсивний алгоритм «розділяй і володарюй» із програмним контролем глибини стека та порівняти його з ітеративним аналогом.

1.1. Індивідуальне завдання (варіант 2)

Частина	**Умова**
Варіативна функція	калькулятор метрик надійності з *args
Lambda-сортування	вузли за правилом [CPU_Usage (спадання), RAM_Free (зростання)]
Рекурсивний алгоритм	сортування масиву злиттям (Merge Sort)
Граничні умови	рекурсивний поділ навпіл, злиття двох впорядкованих підмасивів

1.2. Математичний опис

Для вузла задано n інтервалів напрацювання між відмовами T_1, …, T_n (години), середній час відновлення MTTR та час місії t. Метрики надійності обчислюються за експоненційною моделлю відмов:

$$MTBF = (1/n) \cdot \Sigma\, T_i, \quad \lambda = 1 / MTBF$$

$$A = MTBF / (MTBF + MTTR), \quad R(t) = e^{(-\lambda \cdot t)}$$

Тут MTBF — середнє напрацювання на відмову, λ — інтенсивність відмов, A — коефіцієнт готовності (частка часу, коли вузол працездатний), R(t) — ймовірність безвідмовної роботи протягом часу t. Річний простій оцінюється як (1 − A) · 8760 год.

Багатокритеріальний порядок вузлів задається ключем сортування — кортежем, який порівнюється лексикографічно:

$$key(v) = (-CPU_Usage(v),\ RAM_Free(v))$$

Знак мінус перед CPU_Usage перетворює сортування за зростанням на сортування за спаданням, а другий компонент впорядковує вузли з однаковим навантаженням за зростанням вільної пам’яті.

Сортування злиттям поділяє масив із n елементів навпіл, рекурсивно сортує половини й зливає їх за лінійний час. Часова складність описується рекурентним співвідношенням, глибина рекурсії та кількість викликів визначаються формулами:

$$T(n) = 2 \cdot T(n/2) + \Theta(n)$$

$$D(n) = \lceil \log_2 n \rceil + 1, \quad \text{кількість викликів} = 2n - 1$$

2. Алгоритмічний аналіз

2.1. Варіативна сигнатура та валідація

Функція calculate_reliability_metrics(*intervals_h, **options) приймає довільну кількість інтервалів напрацювання та іменовані опції. Під час виклику CPython пакує позиційні аргументи в незмінний кортеж, а іменовані — у словник:

Параметр	Тип в тілі	Призначення
*intervals_h	tuple	інтервали напрацювання між відмовами, години
mttr_hours	float	середній час відновлення, за замовчуванням 2,0 год
mission_time_h	float	час місії t для R(t), за замовчуванням 720 год (30 діб)
strict	bool	False: некоректні інтервали пропускаються з попередженням; True: ValueError

Опції читаються методом options.get(ім’я, значення_за_замовчуванням). Оскільки **options приймає будь-які імена, функція окремо перевіряє їх проти множини ALLOWED_OPTIONS і для опечатки (наприклад, mission_tim_h) генерує TypeError. Алгоритм валідації:

1) перевірити імена опцій і додатність mttr_hours та mission_time_h;
2) для кожного інтервалу перевірити, що це число (bool відкидається, бо є підкласом int), скінченне й додатне;
3) некоректний інтервал пропустити з попередженням або, за strict=True, згенерувати ValueError;
4) якщо жодного коректного інтервалу не залишилось, згенерувати ValueError;
5) обчислити MTBF, λ, A, R(t) та річний простій і повернути структурований словник.

2.2. Простори імен і замикання

Пошук імен виконується за правилом LEGB: Local → Enclosing → Global → Built-in. Функція make_call_counter демонструє рівень Enclosing: змінна count створюється в зовнішній функції, а внутрішня функція increment змінює її через оператор nonlocal, не виходячи в глобальний простір. Лічильник підраховує кількість виконаних обчислень метрик. Без nonlocal присвоєння count += step створило б локальну змінну й призвело б до UnboundLocalError.

2.3. Lambda-сортування та функції вищого порядку

Вузли впорядковуються викликом sorted(reports, key=lambda r: (−r["cpu_usage"], r["ram_free_gb"])). Функція sorted() виконує стійке сортування (Timsort), а ключ обчислюється один раз для кожного елемента. Для двох вузлів з однаковим CPU (NODE-02 та NODE-03, обидва 76,2 %) порядок визначається другим компонентом кортежу: перший іде вузол із меншою вільною пам’яттю. Додатково застосовано filter(lambda …) для відбору вузлів із CPU > 80 %, map(lambda …) для вилучення їхніх

ідентифікаторів та min(…, key=lambda …) для пошуку вузла з найменшою готовністю.

2.4. Сортування злиттям (Merge Sort)

Алгоритм складається з трьох кроків: поділ масиву навпіл за індексом mid = len // 2, рекурсивне сортування лівої та правої частин, злиття двох впорядкованих підмасивів. Базовий випадок — масив із 0 або 1 елемента: він уже впорядкований, і рекурсія на ньому завершується без подальших викликів, тому кожна гілка гарантовано досягає базового випадку (довжина підмасиву щоразу зменшується приблизно вдвічі). Злиття виконується двома вказівниками; умова left[i] <= right[j] робить сортування стійким.

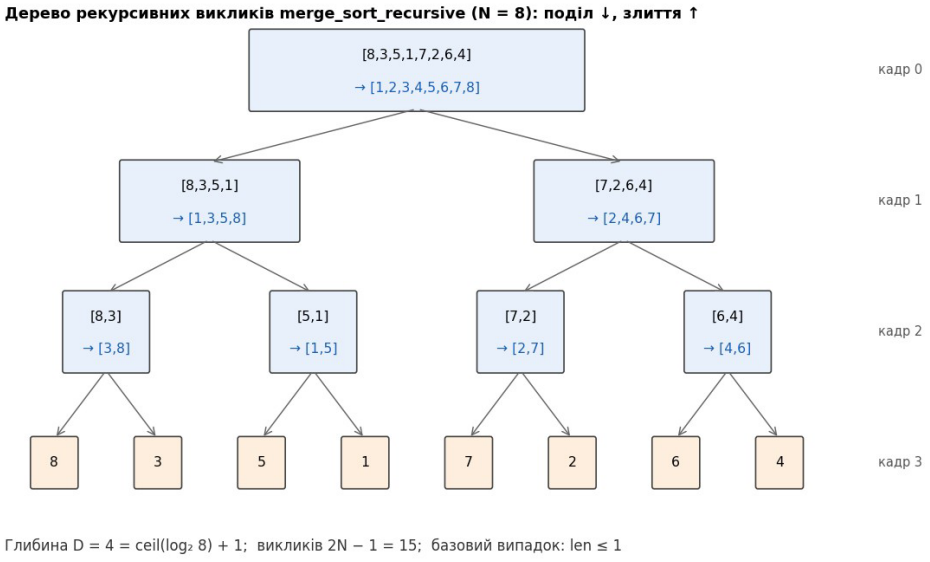


Рисунок 1 — Дерево рекурсивних викликів Merge Sort для масиву з 8 елементів

За основною теоремою про рекурентні співвідношення (Master Theorem) для T(n) = a · T(n/b) + f(n) маємо a = 2, b = 2, f(n) = Θ(n). Порівнюємо f(n) з n^(log_b a) = n^(log₂ 2) = n. Функції збігаються за порядком, тому виконується випадок 2 теореми, і T(n) = Θ(n · log n). Ця оцінка не залежить від вхідних даних: сортування злиттям має складність Θ(n log n) і в найкращому, і в середньому, і в найгіршому випадках.

Характеристика	Merge Sort	Примітка
Час (найкращий / середній / найгірший)	Θ(n log n)	не залежить від даних
Додаткова пам’ять	O(n)	буфери під час злиття
Глибина рекурсії	O(log n)	D = ⌈log₂ n⌉ + 1
Кількість викликів	2n − 1	n листків + (n − 1) внутрішніх вузлів
Порівнянь (верхня оцінка)	n⌈log₂ n⌉ − 2^⌈log₂ n⌉ + 1	для n = 20: 69
Стійкість	так	умова left[i] <= right[j]

Захист від переповнення стека реалізовано класом RecursionDepthTracker: кожен рекурсивний виклик викликає enter(), що збільшує поточну глибину й перевіряє безпечний ліміт, а блок finally гарантує виклик leave() навіть при винятку. При перевищенні ліміту генерується RecursionError. Оскільки глибина Merge Sort логарифмічна (для n = 10^6 лише D = 21), стандартний ліміт CPython у 1000 кадрів недосяжний на практиці — на відміну від лінійної рекурсії, де глибина дорівнює n.

Ітеративний аналог merge_sort_iterative працює «знизу вгору»: спочатку зливає сусідні відрізки довжини 1, потім 2, 4, 8 тощо, використовуючи два буфери однакового розміру. Він має ту саму асимптотику Θ(n log n), але не створює стекових кадрів і не копіює зрізи при кожному виклику.

3. Програмний код

3.1. Файл functional_pipeline.py

```
"""
Лабораторна робота № 7. Модуль 1. Варіант 2.
Калькулятор метрик надійності вузлів із варіативною сигнатурою (*args, **kwargs),
валідацією вхідних даних та багатокритеріальне lambda-сортування вузлів
за правилом [CPU_Usage (спадання), RAM_Free (зростання)].

Метрики надійності за експоненційною моделлю відмов:
    MTBF = (1/n) * sum(T_i)            середнє напрацювання на відмову, год
    A    = MTBF / (MTBF + MTTR)        коефіцієнт готовності
    lam  = 1 / MTBF                    інтенсивність відмов, 1/год
```

```
    R(t) = exp(-lam * t)              ймовірність безвідмовної роботи за час t
Освітній компонент: Програмування на Python.
"""

import math
import time
from typing import Any, Callable

REPORT_WIDTH: int = 88
HOURS_PER_YEAR: float = 8760.0
ALLOWED_OPTIONS: frozenset[str] = frozenset({"mttr_hours", "mission_time_h",
"strict"})


def calculate_reliability_metrics(*intervals_h: float, **options: Any) ->
dict[str, float]:
    """
    Обчислює метрики надійності вузла.
    *intervals_h - довільна кількість інтервалів напрацювання між відмовами
(години).
    **options    - опції: mttr_hours (за замовчуванням 2.0), mission_time_h
(720.0),
                   strict (False: некоректні інтервали пропускаються, True:
викликають ValueError).
    Повертає структурований словник метрик.
    """
    # Перевірка імен опцій: **options приймає будь-які імена, тому опечатки
ловимо вручну
    unknown = set(options) - ALLOWED_OPTIONS
    if unknown:
        raise TypeError(f"Невідомі опції: {sorted(unknown)}")

    # Значення за замовчуванням
    mttr_hours = options.get("mttr_hours", 2.0)
    mission_time_h = options.get("mission_time_h", 720.0)
    strict = options.get("strict", False)

    if mttr_hours <= 0 or mission_time_h <= 0:
        raise ValueError("mttr_hours та mission_time_h повинні бути строго
додатними.")

    valid: list[float] = []
    for idx, value in enumerate(intervals_h, start=1):
        # bool є підкласом int, тому True/False відкидаємо окремо
        is_number = isinstance(value, (int, float)) and not isinstance(value,
bool)
        if not is_number or not math.isfinite(value) or value <= 0:
            if strict:
                raise ValueError(f"Інтервал #{idx} ({value!r}) некоректний.")
            print(f"Попередження: інтервал #{idx} ({value!r}) некоректний і
пропущений.")
```

```
            continue
        valid.append(float(value))

    if not valid:
        raise ValueError("Немає жодного коректного інтервалу напрацювання.")

    mtbf = sum(valid) / len(valid)
    failure_rate = 1.0 / mtbf
    availability = mtbf / (mtbf + mttr_hours)
    reliability = math.exp(-failure_rate * mission_time_h)

    return {
        "samples": len(valid),
        "skipped": len(intervals_h) - len(valid),
        "mtbf_h": mtbf,
        "mttr_h": float(mttr_hours),
        "failure_rate": failure_rate,
        "availability": availability,
        "reliability": reliability,
        "downtime_year_h": (1.0 - availability) * HOURS_PER_YEAR,
    }


def make_call_counter() -> Callable[[], int]:
    """
    Замикання (Enclosing scope): змінна count живе в області видимості
make_call_counter,
    а оператор nonlocal дозволяє внутрішній функції змінювати її без виходу в
глобальний простір.
    """
    count = 0

    def increment(step: int = 1) -> int:
        """Збільшує лічильник на step (0 - лише прочитати поточне значення)."""
        nonlocal count
        count += step
        return count

    return increment


def build_node_reports(nodes: list[tuple], mission_time_h: float) -> list[dict]:
    """
    Для кожного вузла викликає калькулятор метрик, розпаковуючи кортеж інтервалів
через *
    та передаючи опції іменованими аргументами. Вузли без валідних даних
виключаються.
    Формат вузла: (node_id, cpu_usage, ram_free_gb, mttr_hours, intervals_tuple).
    """
    tick = make_call_counter()
    reports = []
```

```
    for node_id, cpu_usage, ram_free_gb, mttr, intervals in nodes:
        try:
            metrics = calculate_reliability_metrics(
                *intervals, mttr_hours=mttr, mission_time_h=mission_time_h
            )
        except ValueError as err:
            print(f"Вузол {node_id} виключено з аналізу: {err}")
            continue
        tick()
        reports.append({"node_id": node_id, "cpu_usage": cpu_usage,
                        "ram_free_gb": ram_free_gb, **metrics})
    print(f"Виконано обчислень метрик (nonlocal-лічильник): {tick(0)}")
    return reports


def print_report_table(sorted_reports: list[dict]) -> None:
    """Виводить таблицю вузлів, відсортованих за [CPU desc, RAM_Free asc]."""
    header = (f"| {'№':^3} | {'Вузол':^11} | {'CPU, %':^7} | {'RAM, ГБ':^9} |
{'MTBF, год':^9} | "
              f"{'MTTR':^6} | {'A, %':^9} | {'R(t), %':^9} |")
    print("=" * REPORT_WIDTH)
    print(f"{'МЕТРИКИ НАДІЙНОСТІ ВУЗЛІВ (СОРТУВАННЯ: CPU ↓, RAM_FREE
↑)':^{REPORT_WIDTH}}")
    print("=" * REPORT_WIDTH)
    print(header)
    print("-" * REPORT_WIDTH)
    for idx, rec in enumerate(sorted_reports, start=1):
        print(f"| {idx:^3d} | {rec['node_id']:<11} | {rec['cpu_usage']:^7.1f} | "
              f"{rec['ram_free_gb']:^9.1f} | {rec['mtbf_h']:^9.1f} |
{rec['mttr_h']:^6.1f} | "
              f"{rec['availability'] * 100:^9.3f} | {rec['reliability'] *
100:^9.2f} |")
    print("=" * REPORT_WIDTH)


def main() -> None:
    """Головна функція демонстрації варіативних сигнатур та lambda-сортування."""
    t_start = time.perf_counter()
    mission_time_h = 720.0   # місія: 30 діб безперервної роботи

    # (node_id, CPU_Usage %, RAM_Free ГБ, MTTR год, інтервали напрацювання між
відмовами, год)
    raw_nodes = [
        ("NODE-01", 91.4, 1.8, 2.0, (702.0, 815.5, 640.0, 910.2, 788.4)),
        ("NODE-02", 76.2, 6.4, 3.0, (1200.0, 1350.5, 980.0)),
        ("NODE-03", 76.2, 3.2, 1.5, (450.0, 520.5, 610.0, 380.0)),      # CPU як
у NODE-02
        ("NODE-04", 55.0, 12.0, 4.0, (2100.0, 1850.0, 2400.5)),
        ("NODE-05", 83.7, 4.5, 2.5, (300.0, -50.0, 410.0, "abc", 280.0)),  # два
некоректні
```

```
        ("NODE-06", 33.3, 14.2, 2.0, ()),                                  # немає
даних
        ("NODE-07", 68.9, 8.0, 2.0, (1500.0,)),
    ]

    print("=" * REPORT_WIDTH)
    print(f"{'ВАРІАНТ 2. КАЛЬКУЛЯТОР МЕТРИК НАДІЙНОСТІ ТА LAMBDA-
СОРТУВАННЯ':^{REPORT_WIDTH}}")
    print("=" * REPORT_WIDTH)
    print(f"Вхідних вузлів: {len(raw_nodes)}, час місії t = {mission_time_h:g}
год\n")

    # 1. Обчислення метрик (*args через розпакування кортежу, **kwargs через
іменовані опції)
    reports = build_node_reports(raw_nodes, mission_time_h)
    print(f"Вузлів у звіті: {len(reports)}\n")

    # 2. Багатокритеріальне сортування: CPU за спаданням, RAM_Free за зростанням
    sorted_reports = sorted(reports, key=lambda r: (-r["cpu_usage"],
r["ram_free_gb"]))
    print_report_table(sorted_reports)

    # 3. Функції вищого порядку: filter, map, min з lambda
    overloaded = list(filter(lambda r: r["cpu_usage"] > 80.0, sorted_reports))
    overloaded_ids = list(map(lambda r: r["node_id"], overloaded))
    weakest = min(reports, key=lambda r: r["availability"])
    print(f"Вузли з CPU > 80 % (filter + map):        {overloaded_ids}")
    print(f"Найменша готовність A (min + lambda):     {weakest['node_id']} "
          f"({weakest['availability'] * 100:.3f}%)")
    print(f"Простій найгіршого вузла за рік:
 {weakest['downtime_year_h']:.1f} год")

    # 4. Захист від опечаток в іменах опцій
    try:
        calculate_reliability_metrics(100.0, 120.0, mission_tim_h=500.0)
    except TypeError as err:
        print(f"Перехоплено TypeError: {err}")

    # 5. Режим strict=True перетворює некоректний інтервал на виняток
    try:
        calculate_reliability_metrics(100.0, -5.0, strict=True)
    except ValueError as err:
        print(f"Перехоплено ValueError (strict=True): {err}")

    elapsed = time.perf_counter() - t_start
    print(f"\nЧас виконання функціонального конвеєра: {elapsed * 1000:.3f} мс\n")


if __name__ == "__main__":
    main()
```

3.2. Файл recursive_algorithms.py

```
"""
Лабораторна робота № 7. Модуль 2. Варіант 2.
Рекурсивне сортування злиттям (Merge Sort): рекурсивний поділ масиву навпіл
та злиття двох впорядкованих підмасивів. Програмний контроль глибини стека,
порівняння з ітеративним (висхідним) аналогом і з вбудованим Timsort.

Рекурентне співвідношення: T(n) = 2 * T(n / 2) + Theta(n)  =>  Theta(n log n)
Глибина рекурсії: D(n) = ceil(log2 n) + 1
Освітній компонент: Програмування на Python.
"""

import random
import sys
import time

REPORT_WIDTH: int = 88


class RecursionDepthTracker:
    """Моніторинг та захист глибини рекурсивного стека; також рахує виклики й
порівняння."""

    def __init__(self, max_allowed_depth: int = 500) -> None:
        self.current_depth = 0
        self.max_observed_depth = 0
        self.max_allowed_depth = max_allowed_depth
        self.calls = 0
        self.comparisons = 0

    def enter(self) -> None:
        """Фіксує вхід у новий рекурсивний кадр і перевіряє безпечний ліміт."""
        self.current_depth += 1
        self.calls += 1
        self.max_observed_depth = max(self.max_observed_depth,
self.current_depth)
        if self.current_depth > self.max_allowed_depth:
            raise RecursionError(
                f"Перевищено безпечний ліміт глибини: {self.current_depth} >
{self.max_allowed_depth}"
            )

    def leave(self) -> None:
        """Фіксує вихід зі стекового кадру."""
        self.current_depth -= 1


def merge(left: list[int], right: list[int], tracker: RecursionDepthTracker |
None = None) -> list[int]:
    """
```

```
    Зливає два впорядковані підмасиви в один за O(len(left) + len(right)).
    Умова left[i] <= right[j] зберігає стійкість сортування (рівні елементи не
міняються місцями).
    """
    merged: list[int] = []
    i = j = 0
    while i < len(left) and j < len(right):
        if tracker is not None:
            tracker.comparisons += 1
        if left[i] <= right[j]:
            merged.append(left[i])
            i += 1
        else:
            merged.append(right[j])
            j += 1
    # Один із підмасивів вичерпано: решту іншого дописуємо без порівнянь
    merged.extend(left[i:])
    merged.extend(right[j:])
    return merged


def merge_sort_recursive(arr: list[int], tracker: RecursionDepthTracker) ->
list[int]:
    """Рекурсивне сортування злиттям з обліком глибини стека."""
    tracker.enter()
    try:
        # Базовий випадок: масив з 0 або 1 елемента вже впорядкований
        if len(arr) <= 1:
            return list(arr)

        mid = len(arr) // 2
        left_sorted = merge_sort_recursive(arr[:mid], tracker)    # лівий
підмасив
        right_sorted = merge_sort_recursive(arr[mid:], tracker)   # правий
підмасив
        return merge(left_sorted, right_sorted, tracker)
    finally:
        tracker.leave()


def merge_sort_iterative(arr: list[int]) -> list[int]:
    """
    Ітеративне висхідне (bottom-up) сортування злиттям без рекурсії.
    Спочатку зливаються сусідні відрізки довжини 1, потім 2, 4, 8 тощо.
    Використовує два буфери однакового розміру, глибина стека константна.
    """
    n = len(arr)
    src, dst = list(arr), [0] * n
    width = 1
    while width < n:
        for lo in range(0, n, 2 * width):
```

```
            mid, hi = min(lo + width, n), min(lo + 2 * width, n)
            i, j, k = lo, mid, lo
            while i < mid and j < hi:
                if src[i] <= src[j]:
                    dst[k] = src[i]
                    i += 1
                else:
                    dst[k] = src[j]
                    j += 1
                k += 1
            dst[k:hi] = src[i:mid] + src[j:hi]   # одна з частин порожня
        src, dst = dst, src
        width *= 2
    return src


def theoretical_depth(n: int) -> int:
    """Теоретична глибина рекурсії D(n) = ceil(log2 n) + 1 (для n >= 1)."""
    return (n - 1).bit_length() + 1


def recursive_sum(n: int) -> int:
    """Лінійна рекурсія глибиною n: приклад небезпечної рекурсії, що вичерпує
стек."""
    return 0 if n == 0 else n + recursive_sum(n - 1)


def best_time_ms(func, data: list[int], repeats: int = 3) -> float:
    """Повертає найкращий з repeats замірів часу виконання func(data) у
мілісекундах."""
    best = float("inf")
    for _ in range(repeats):
        t0 = time.perf_counter()
        func(data)
        best = min(best, time.perf_counter() - t0)
    return best * 1000.0


def demo_small_array() -> None:
    """Демонстрація на масиві з 20 елементів із моніторингом стека."""
    print(f"Системний ліміт глибини стека CPython: {sys.getrecursionlimit()}
кадрів")
    random.seed(2026)
    test_array = [random.randint(-100, 100) for _ in range(20)]
    print(f"Початковий масив (N = {len(test_array)}):   {test_array}")
    print("-" * REPORT_WIDTH)

    tracker = RecursionDepthTracker(max_allowed_depth=200)
    result = merge_sort_recursive(test_array, tracker)
    print(f"Відсортований масив:               {result}")
```

```
    print(f"Верифікація (== sorted):          {str(result ==
sorted(test_array)).upper()}")
    print(f"Ітеративний аналог збігається:
{str(merge_sort_iterative(test_array) == result).upper()}")
    print(f"Максимальна глибина D:            {tracker.max_observed_depth}
кадрів "
          f"(теорія: {theoretical_depth(len(test_array))})")
    print(f"Рекурсивних викликів:             {tracker.calls} (теорія: 2N - 1 =
{2 * len(test_array) - 1})")
    print(f"Порівнянь елементів у злиттях:    {tracker.comparisons}")
    print("-" * REPORT_WIDTH)

    # Захист від переповнення: штучно низький безпечний ліміт
    strict_tracker = RecursionDepthTracker(max_allowed_depth=4)
    try:
        merge_sort_recursive(test_array, strict_tracker)
    except RecursionError as err:
        print(f"Перехоплено RecursionError (ліміт 4): {err}")

    # Для порівняння: лінійна рекурсія глибиною 5000 вичерпує системний стек
    try:
        recursive_sum(5000)
    except RecursionError:
        print(f"recursive_sum(5000) -> RecursionError: глибина перевищила
{sys.getrecursionlimit()}")
    print(f"Merge Sort для N = 1 000 000 потребує лише D =
{theoretical_depth(1_000_000)} кадрів")


def demo_benchmark() -> None:
    """Порівняння швидкодії рекурсивного, ітеративного сортування злиттям та
sorted()."""
    print(f"\n{'ПОРІВНЯННЯ ШВИДКОДІЇ (найкращий із 3 замірів)':^{REPORT_WIDTH}}")
    print("-" * REPORT_WIDTH)
    print(f"| {'N':^8} | {'Рекурс., мс':^12} | {'Ітерат., мс':^12} | {'sorted,
мс':^12} | "
          f"{'D':^5} | {'D теор.':^7} | {'Порівнянь':^10} |")
    print("-" * REPORT_WIDTH)

    random.seed(7)
    for n in (1_000, 5_000, 20_000, 100_000):
        data = [random.randint(-10**6, 10**6) for _ in range(n)]

        tracker = RecursionDepthTracker(max_allowed_depth=100)
        rec_result = merge_sort_recursive(data, tracker)
        assert rec_result == sorted(data) == merge_sort_iterative(data), "Помилка
сортування!"

        t_rec = best_time_ms(lambda d: merge_sort_recursive(d,
RecursionDepthTracker(100)), data)
        t_iter = best_time_ms(merge_sort_iterative, data)
```

```
        t_builtin = best_time_ms(sorted, data)

        print(f"| {n:>8d} | {t_rec:>12.3f} | {t_iter:>12.3f} | {t_builtin:>12.3f}
| "
              f"{tracker.max_observed_depth:^5d} | {theoretical_depth(n):^7d} |
{tracker.comparisons:>10d} |")
    print("=" * REPORT_WIDTH + "\n")


def main() -> None:
    """Головна точка входу: демонстрація та профілювання сортування злиттям."""
    print("=" * REPORT_WIDTH)
    print(f"{'ВАРІАНТ 2. РЕКУРСИВНЕ СОРТУВАННЯ ЗЛИТТЯМ (MERGE
SORT)':^{REPORT_WIDTH}}")
    print("=" * REPORT_WIDTH)
    demo_small_array()
    demo_benchmark()


if __name__ == "__main__":
    main()
```

4. Результати тестування

Запуск виконувався в середовищі Conda ce_lab_env командами python functional_pipeline.py та python recursive_algorithms.py.

4.1. Результат роботи functional_pipeline.py

```
PS C:\Users\Janis\Desktop\урок\Програмування на Python. Базовий рівень\lab7> (C:\Users\Janis\
miniconda3\Scripts\activate) ; (conda activate ce_lab_env)
PS C:\Users\Janis\Desktop\урок\Програмування на Python. Базовий рівень\lab7> & C:\Users\Janis
\miniconda3\envs\ce_lab_env\python.exe "c:/Users/Janis/Desktop/урок/Програмування на Python.
Базовий рівень/lab7/functional_pipeline.py"
================================================================================
            ВАРІАНТ 2. КАЛЬКУЛЯТОР МЕТРИК НАДІЙНОСТІ ТА LAMBDA-СОРТУВАННЯ
================================================================================
Вхідних вузлів: 7, час місії t = 720 год

Попередження: інтервал #2 (-50.0) некоректний і пропущений.
Попередження: інтервал #4 ('abc') некоректний і пропущений.
Вузол NODE-06 виключено з аналізу: Немає жодного коректного інтервалу напрацювання.
Виконано обчислень метрик (nonlocal-лічильник): 6
Вузлів у звіті: 6

================================================================================
              МЕТРИКИ НАДІЙНОСТІ ВУЗЛІВ (СОРТУВАННЯ: CPU ↓, RAM_FREE ↑)
================================================================================
|  №  |    Вузол    | CPU, % |  RAM, ГБ  | MTBF, год |  MTTR  |    A, %    |  R(t), %  |
--------------------------------------------------------------------------------
|  1  | NODE-01     |  91.4  |    1.8    |   771.2   |  2.0   |   99.741   |   39.31   |
|  2  | NODE-05     |  83.7  |    4.5    |   330.0   |  2.5   |   99.248   |   11.28   |
|  3  | NODE-03     |  76.2  |    3.2    |   490.1   |  1.5   |   99.695   |   23.02   |
|  4  | NODE-02     |  76.2  |    6.4    |  1176.8   |  3.0   |   99.746   |   54.24   |
|  5  | NODE-07     |  68.9  |    8.0    |  1500.0   |  2.0   |   99.867   |   61.88   |
|  6  | NODE-04     |  55.0  |   12.0    |  2116.8   |  4.0   |   99.811   |   71.17   |
================================================================================
Вузли з CPU > 80 % (filter + map):        ['NODE-01', 'NODE-05']
Найменша готовність A (min + lambda):     NODE-05 (99.248%)
Простій найгіршого вузла за рік:          65.9 год
Перехоплено TypeError: Невідомі опції: ['mission_tim_h']
Перехоплено ValueError (strict=True): Інтервал #2 (-5.0) некоректний.

Час виконання функціонального конвеєра: 3.528 мс
```

Скріншот 1 — Метрики надійності вузлів і результат lambda-сортування

З 7 вхідних вузлів у звіті залишилось 6. Для NODE-05 пропущено два некоректні інтервали (від'ємне −50,0 та рядок "abc"), решта три (300, 410 та 280 год) дали MTBF = 330,0 год; вузол NODE-06 не має даних і виключений за ValueError. Лічильник на замиканні зафіксував 6 виконаних обчислень.

Перевірка вручну для NODE-01: MTBF = (702,0 + 815,5 + 640,0 + 910,2 + 788,4) / 5 = 771,2 год; A = 771,2 / (771,2 + 2,0) = 99,741 %; R(720) = e^(−720/771,2) = 39,31 %. Порядок у таблиці відповідає правилу [CPU ↓, RAM_Free ↑]: NODE-01 (91,4 %), NODE-05 (83,7 %), далі NODE-03 та NODE-02 з однаковим CPU 76,2 %, але RAM_Free 3,2 ГБ і 6,4 ГБ відповідно,

потім NODE-07 і NODE-04. Фільтр знайшов два вузли з CPU > 80 % (NODE-01 та NODE-05). Найменшу готовність має NODE-05 (99,248 %), що відповідає річному простою близько 65,9 год. Опечатка в імені опції та режим strict=True коректно перехоплені як TypeError і ValueError.

4.2. Результат роботи recursive_algorithms.py

```
PS C:\Users\Janis\Desktop\ypok\Програмування на Python. Базовий рівень\lab7> (C:\Users\Janis\miniconda3\Scripts\activate) ;
(conda activate ce_lab_env)
PS C:\Users\Janis\Desktop\ypok\Програмування на Python. Базовий рівень\lab7> & C:\Users\Janis\miniconda3\envs\ce_lab_env\pyt
hon.exe "c:/Users/Janis/Desktop/ypok/Програмування на Python. Базовий рівень/lab7/recursive_algorithms.py"
================================================================================
                ВАРІАНТ 2. РЕКУРСИВНЕ СОРТУВАННЯ ЗЛИТТЯМ (MERGE SORT)
================================================================================
Системний ліміт глибини стека CPython: 1000 кадрів
Початковий масив (N = 20):   [-70, -19, 28, 31, 65, -74, -43, 53, 59, 42, 7, 100, 46, 40, 87, 98, 96, 25, 92, 97]
--------------------------------------------------------------------------------
Відсортований масив:              [-74, -70, -43, -19, 7, 25, 28, 31, 40, 42, 46, 53, 59, 65, 87, 92, 96, 97, 98, 100]
Верифікація (== sorted):          TRUE
Ітеративний аналог збігається:    TRUE
Максимальна глибина D:            6 кадрів (теорія: 6)
Рекурсивних викликів:             39 (теорія: 2N - 1 = 39)
Порівнянь елементів у злиттях:    58
--------------------------------------------------------------------------------
Перехоплено RecursionError (ліміт 4): Перевищено безпечний ліміт глибини: 5 > 4
recursive_sum(5000) -> RecursionError: глибина перевищила 1000
Merge Sort для N = 1 000 000 потребує лише D = 21 кадрів

                    ПОРІВНЯННЯ ШВИДКОДІЇ (найкращий із 3 замірів)
--------------------------------------------------------------------------------
|    N     | Рекурс., мс  | Ітерат., мс  |  sorted, мс  |   D   | D теор. | Порівнянь  |
--------------------------------------------------------------------------------
|     1000 |        7.449 |        3.460 |        0.059 |  11   |   11    |       8697 |
|     5000 |       44.773 |       21.942 |        0.412 |  14   |   14    |      55277 |
|    20000 |      200.343 |      110.711 |        2.131 |  16   |   16    |     260849 |
|   100000 |     1150.439 |      600.501 |       14.828 |  18   |   18    |    1536398 |
================================================================================
```

Скріншот 2 — Сортування злиттям: моніторинг стека та порівняння швидкодії

Для масиву з 20 елементів зафіксовано максимальну глибину рекурсії 6 кадрів, що збігається з теоретичним $D = \lceil \log_2 20 \rceil + 1 = 5 + 1 = 6$. Кількість викликів 39 дорівнює $2N - 1$. Кількість порівнянь 58 не перевищує верхню оцінку $20 \cdot 5 - 32 + 1 = 69$. Штучно занижений ліміт глибини 4 призвів до RecursionError, а лінійна рекурсія recursive_sum(5000) вичерпала стандартний ліміт 1000 кадрів. Для $N = 10^6$ Merge Sort потребує лише 21 кадр.

У тесті швидкодії для N від 1000 до 100 000 теоретична й фактична глибина збігаються в усіх рядках (11, 14, 16, 18). Ітеративна версія швидша за рекурсивну приблизно в 1,6–1,75 раза, оскільки не витрачає час на виклики функцій, відстеження глибини та створення зрізів. Вбудований sorted() швидший за рекурсивну версію приблизно в 15 раз при N = 100 000, адже

Timsort реалізований мовою C. Час зростає трохи швидше за лінійний, що відповідає Θ(n log n).

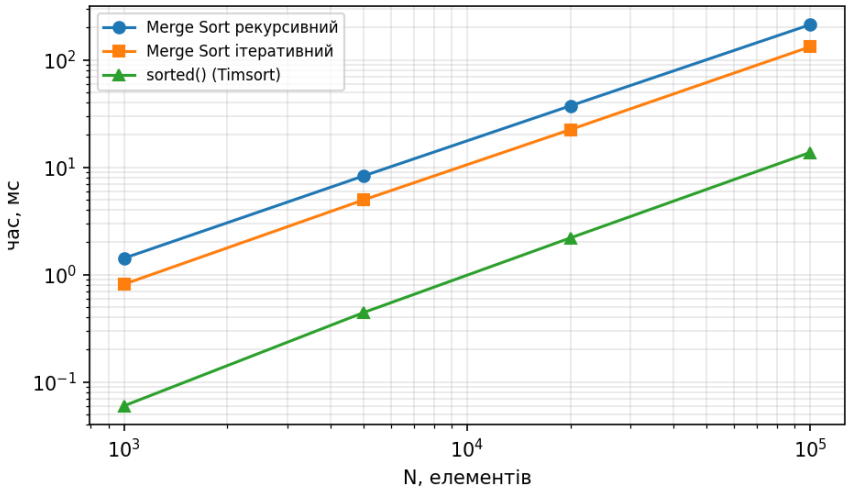


Рисунок 4 — Залежність часу сортування від N (логарифмічні осі)

5. Посилання на віддалений репозиторій

Профіль: https://github.com/kefir4ikk

Репозиторій лабораторної роботи :
https://github.com/kefir4ikk/progPython_YevgeniiYatsentiuk/tree/master/labs/lab7

6. Висновки

У ході лабораторної роботи реалізовано калькулятор метрик надійності з варіативною сигнатурою та алгоритм сортування злиттям для варіанта 2. Пакування аргументів у кортеж (*args) і словник (**kwargs) дозволило створити гнучкий інтерфейс функції зі значеннями за замовчуванням, валідацією даних і контролем імен опцій. Правило LEGB і оператор nonlocal продемонстровано на замиканні-лічильнику.

Lambda-вираз із кортежем (−CPU_Usage, RAM_Free) у ролі ключа sorted() дав стійке багатокритеріальне сортування в один рядок, а filter, map та min із lambda забезпечили компактну аналітику без явних циклів. Обмеження lambda (лише один вираз) не завадило задачі, а складніша логіка залишилась у функціях def.

Рекурсивний Merge Sort має глибину $D = \lceil \log_2 n \rceil + 1$, що експериментально підтверджено в усіх тестах, тому просторові накладні витрати стека логарифмічні й переповнення стека не загрожує навіть для мільйона елементів; лінійна рекурсія натомість вичерпує ліміт у 1000 кадрів. Ітеративний висхідний варіант з тією ж складністю $\Theta(n \log n)$ виявився швидшим за рекурсивний у 1,6–1,75 раза завдяки відсутності накладних витрат на виклики, але рекурсивна версія простіша для розуміння й безпосередньо відображає схему «розділяй і володарюй». Вбудований sorted() значно випереджає обидві версії завдяки реалізації на C та адаптивності Timsort, тому в практичних задачах слід використовувати саме його, а власні реалізації доцільні для навчання й особливих вимог до алгоритму.

Контрольні питання:

1. Як влаштовано механізм передачі параметрів pass-by-object-reference у CPython? Чому спроба змінити значення цілочисельного аргументу всередині функції не впливає на вихідну змінну, а зміна списку через метод append() мутує вихідний об'єкт?

У Python функція отримує не копію значення й не посилання на змінну, а ім’я, прив’язане до того самого об’єкта, що й змінна у викликача. Ціле число незмінне: вираз x += 1 створює новий об’єкт і лише перепризначає локальне ім’я, а ім’я у викликача лишається прив’язаним до старого об’єкта. Список змінний: метод append() змінює сам об’єкт на місці, і ця зміна видима через усі посилання на нього. Перепризначення lst = lst + [1] не вплине на викликача, тоді як lst += [1] (метод __iadd__) мутує список на місці.

2. Опишіть послідовність пошуку ідентифікаторів згідно з правилом LEGB. За яких умов інтерпретатор генерує виняток UnboundLocalError при зверненні до змінної всередині функції?

Інтерпретатор шукає ім’я послідовно в просторах: Local (поточна функція), Enclosing (охоплюючі функції), Global (модуль), Built-in (вбудовані імена). UnboundLocalError виникає, коли ім’я присвоюється десь у тілі функції

(компілятор тоді вважає його локальним у всій функції), але читається до присвоєння, наприклад counter += 1 для глобальної змінної без оголошення global. Виправляють оголошеннями global чи nonlocal або передаванням значення як параметра.

3. У чому полягає різниця між операторами *args та **kwargs у сигнатурі функції? До яких типів даних CPython перетворює передані аргументи під час пакування?

*args збирає зайві позиційні аргументи в незмінний кортеж tuple, **kwargs збирає зайві іменовані аргументи в словник dict, який зберігає порядок додавання. Ті самі символи при виклику розпаковують колекцію: f(*items, **opts). Порядок параметрів у сигнатурі: звичайні, *args, параметри лише за іменем, **kwargs.

4. Розкрийте системний механізм функціонування lambda-виразів у Python. Які синтаксичні та функціональні обмеження накладаються на lambda-функції порівняно зі стандартними функціями def?

Lambda створює звичайний функціональний об’єкт з іменем <lambda> із тілом, що складається з одного виразу, значення якого повертається автоматично. Вона підтримує замикання й значення за замовчуванням. Обмеження: тіло може містити лише вираз (без операторів присвоєння, return, цикли, try; умовний вираз x if c else y допустимий), немає рядка документації та анотацій, а в трасуванні помилок вона анонімна, що ускладнює налагодження. Крім того, lambda в циклі пізно прив’язує змінні замикання, тому часто використовують значення за замовчуванням lambda x, i=i: ….

5. Чому при глибокій рекурсії виникає виняток RecursionError? Які функції модуля sys дозволяють інспектувати та програмно модифікувати ліміт глибини стекових кадрів?

CPython обмежує глибину стека викликів Python-функцій (за замовчуванням 1000 кадрів), щоб глибока чи нескінченна рекурсія не вичерпала стек процесу й не призвела до аварійного завершення. Коли глибина

перевищує ліміт, генерується RecursionError. Поточний ліміт показує sys.getrecursionlimit(), змінює sys.setrecursionlimit(n). Надмірне збільшення ліміту ризиковане: можливе переповнення системного стека. CPython не оптимізує хвостову рекурсію, тому глибоку рекурсію краще замінювати циклом або явним стеком.

6. Сформулюйте сутність теореми Master Theorem для оцінки складності рекурсивних алгоритмів «розділяй і володарюй». Якою є асимптотична складність алгоритму QuickSort у найкращому, середньому та найгіршому випадках?

Для T(n) = a·T(n/b) + f(n) порівнюють f(n) із n^(log_b a). Випадок 1: f(n) асимптотично менша, і T(n) = Θ(n^(log_b a)). Випадок 2: порядок однаковий, і T(n) = Θ(n^(log_b a) · log n). Випадок 3: f(n) суттєво більша (за умови регулярності), і T(n) = Θ(f(n)). Для Merge Sort a = b = 2, f = Θ(n), тому випадок 2 і Θ(n log n). QuickSort у найкращому та середньому випадках має Θ(n log n) (збалансоване розбиття T(n) = 2T(n/2) + n), а в найгіршому Θ(n^2), коли опорний елемент щоразу найменший чи найбільший і T(n) = T(n−1) + n. Саме тому Merge Sort із гарантованою складністю Θ(n log n) надійніший щодо вхідних даних, але потребує O(n) додаткової пам’яті.